

Pointers and 2D arrays, Strings

Pointers and 2D Arrays

1. Each row of a 2D array can be thought of as a one dimensional array. This is very important fact if we wish to access array elements of a two dimensional array using pointers.
2. `int s[5][2];` can be thought of as setting up a one dimensional array of 5 elements, each of which is a one dimensional array of 2 elements long.
3. We refer to an element of a one dimensional array using a single subscript.
4. Similarly if we can imagine `s` to be a one dimensional array then we can refer to its zeroth element as `s[0]`, more specifically `s[0]` gives the address of zeroth one dimensional array. This fact can be illustrated by the following example:

```
#include<iostream>
using namespace std;
```

```
int main()
```

```
{
```

```
    int s[5][2] = { { 1234, 56 }, { 1212, 33 }, { 1434, 80 }, { 1312, 78 } };
```

```
    int i, j;
```

```
    for (i = 0; i <= 3; i++)
```

```
    {
```

```
        cout << &s[i] << endl;
```

```
    }
```

```
    system("Pause");
```

```
    return 0;
```

```
}
```

C:\Users\win 8.1\documents\visual studio 2013\Project

00A7FB60

00A7FB68

00A7FB70

00A7FB78

Press any key to continue . . . _

The compiler knows how many columns are there in array `s` , since we specified this in the array declaration. So it interprets the expression `s[0]` as `(s+0)` , and `s[1]` as `(s+1)`.

Therefore, `(s+0)` is interpreted as address `00A7FB60`

Therefore, `(s+1)` is interpreted as address `00A7FB68`

And so on.

Now we have been able to reach each individual row .

What remains is to be able to refer to individual element of a row.

Suppose we want to refer `s[2][1]` using pointers.

We know that `s[2]` will give us the address of first element in the third row.(address of second one dimensional array)obviously
(`s[2]+1`) would give us the address of second element in the third row,
and the value at this address can be obtained by using the value at the
address operator, `*(s[2]+1)`.

We know that if

```
int num[10];
```

Then `num[i]` is same as `*(num+i)`.

Similarly, `*(s[2]+1)` is same as `*(*(s+2)+1)`.

Thus,

All the following expressions refer to the same element,

```
s[2][1];
```

```
*(*(s+2)+1);
```

```
*(s[2]+1);
```

```
#include<iostream>
using namespace std;
int main()
{
    int a[2][3];
    int i, j;
    cout << "Enter the elements of the array:" << endl;
    for (i = 0; i < 2; i++)
    {
        for (j = 0; j < 3; j++)
        {
            cin >> (*(a + i) + j);
        }
    }

    for (i = 0; i < 2; i++)
    {
        for (j = 0; j < 3; j++)
        {
            cout<<*(a + i) + j)<<"\t";
        }
        cout << endl;
    }

    system("Pause");
    return 0;
}
```

Pointers and 2D Arrays

A 2D array is not the same as an array of pointers to 1D arrays. Actually a 2D array is declared as:

```
int (*ptr)[10];
```

Here, ptr is a pointer to an array of 10 elements .

Note: `int (*ptr)[10]` ; declares ptr to be a pointer to an array of 10 integers. This is different from `int *ptr[10]` ; which would make ptr the name of an array of 10 pointers to type int.

```
#include<iostream>
using namespace std;
int main()
{
    int arr[3][2] = { 1, 2, 3, 4, 5, 6 };
    int(*ptr)[2];
    ptr = arr;
    cout << ptr << endl;
    cout << &arr[0][0]<<endl;
    cout << (*ptr) << endl;
    cout << ptr+1 << endl;
    cout << &arr[1][0] << endl;
    cout << *(ptr+1) << endl;
    system("Pause");
    return 0;
}
```


Strings in C++

Character arrays are many time called strings. strings are used by programming languages to manipulate text such as word or sentences.

A string constant is a one dimensional array of characters terminated by a null(‘\0’).

For example:

```
char name[]="Programming";
```

```
#include<iostream>
using namespace std;
int main()
{
    char name[15] = "Final Exams";
    cout << name << endl;
    system("Pause");
    return 0;
}
```

```
#include<iostream>
using namespace std;
int main()
{
    char name[15] = "Final Exams";
    for (int i = 0; i < 15; i++)
    {
        cout << name[i];
    }
    system("Pause");
    return 0;
}
```

```
#include<iostream>
using namespace std;
```

```
int main()
{
    char name[15] = "Final Exams";
    char *ptr;
    ptr = name;
    int i = 0;
    while (*ptr != '\0')
    {
        cout << *ptr;
        ptr++;
    }
    system("Pause");
    return 0;
}
```

```
#include<iostream>
using namespace std;
int main()
{
    char name[15] ;
    cout << "Enter the String" << endl;
    cin >> name;
    cout << name;
    system("Pause");
    return 0;
}
```

Notice that, in the given example only "Final" is displayed instead of "Final Exam".

This is because the extraction operator >> considers a space " " has a terminating character.

C:\Users\win 8.1\documents\visual studio 2013\Projects\Project9\Debug\Projec...

```
Enter the String
Final Exams
Final
Press any key to continue . . .
```

C++ String to read a line of text

```
#include<iostream>
using namespace std;
int main()
{
    char name[100] ;
    cout << "Enter the String" << endl;
    cin.get(name, 100);
    cout << "You entered: " << name << endl;
    system("Pause");
    return 0;
}
```

String Object

1. In C++, you can also create a string object for holding strings.
2. Unlike using char arrays, string objects has no fixed length, and can be extended as per your requirement.

```
#include<iostream>
#include<string>
using namespace std;
int main()
{
    string name ;
    cout << "Enter the String" << endl;
    getline(cin,name);
    cout << "You entered: " << name << endl;
    system("Pause");
    return 0;
}
```

String Function

1	strcpy(s1, s2): Copies string s2 into string s1.
2	strcat(s1, s2): Concatenates string s2 onto the end of string s1.
3	strlen(s1): Returns the length of string s1.
4	strcmp(s1, s2): Returns 0 if s1 and s2 are the same; less than 0 if $s1 < s2$; greater than 0 if $s1 > s2$.

String Function

```
#include<iostream>
#include<cstring>
using namespace std;
int main()
{
    char subject[16],book[16];
    int l;
    cout << "Enter the String" << endl;
    cin.get( subject,16);
    cout << "Enter the String" << endl;
    cin.get(book,16);
    cout << "You entered: " << subject << endl;
    cout << "You entered: " << book << endl;
    l = strlen(subject);
    cout << "length " << l << endl;
    strcpy_s(subject, book);

    cout << subject << endl;
    strcat_s(subject, book);
    cout << subject<< endl;
    system("Pause");
    return 0;
}
```



```
#include<iostream>
#include<string>
using namespace std;
int main()
{
    string str1 = "Hello";
    string str2 = "World";
    string str3;
    int len;
    // copy str1 into str3
    str3 = str1;
    cout << "str3 : " <<str3 << endl;
    // concatenates str1 and str2
    str3 = str1 + str2;
    cout << "str1 + str2 : " << str3 << endl;
    // total length of str3 after concatenation
    len = str3.size();
    cout << "str3.size() : " << len << endl;
    system("Pause");
    return 0;
}
```